

Rapport de fin de Projet :

Bille sur plateau

**BENOIST Matys
CHOTARD Jules
BA Cire
GAUJAL Martin
Tuteur : Monsieur DUFAY Basile
Année 2024-2025**

Remerciements :

En premier lieu, nous tenons à remercier **Monsieur DUFAY, professeur de l'ESIX** et notre **tuteur** pour son aide et son suivi tout au long de ce projet, ces conseils et son expertise nous ont permis d'avancer plus rapidement. Il nous a beaucoup appris concernant la gestion d'un projet et de ses tâches principales.

Nous profitons de cette occasion pour remercier **Matthias** et **Brice**, nos **camarades de l'ESIX** qui nous ont permis d'effectuer les découpes nécessaires des pièces de notre système. Sans eux, ce projet n'aurait pas pu atteindre l'état actuel.

Nous tenons également à remercier **Monsieur BURGAULT, gestionnaire de la salle projet** qui a commandé dans les plus brefs délais les pièces qui étaient nécessaires à notre projet. Sans lui, nous n'aurions pas pu avancer aussi vite dans notre projet.

Nous tenons également à remercier l'école de l'**ESIX** qui est à la base de ce projet, sans elle, nous n'aurions pas pu progresser techniquement dans la réalisation de tâches de projet, nous n'aurions pas pu progresser d'un point de vue gestion de projet et nous n'aurions pas pu progresser dans notre faculté à travailler en groupe, à se coordonner pour mener à bien un projet. De plus, c'est grâce à l'accès à la salle du Fablab et les outils et machines mis à disposition que nous avons pu avancer dans notre projet.

Sommaire

Projet :	1
<i>Remerciements</i>	2
<i>Liste des figures</i>	4
<i>Introduction</i>	5
<i>Partie Mécanique</i>	6
<i>Partie Électronique</i>	13
<i>Partie Automatique</i>	16
<i>Partie Informatique</i>	21
<i>Difficultés rencontrés</i>	24
<i>Conclusion</i>	25
<i>Annexes</i>	26

Liste des figures :

- Figure 1 : Solution existante
- Figure 2 : Schéma cinématique du système
- Figure 3 : Schéma du système vu de haut
- Figure 4 : Analyse cinématique d'un bras du système
- Figure 5 : Détermination des coordonnées des bras du système
- Figure 6 : Détermination des équations que doit respecter le premier bras
- Figure 7 : Détermination des équations que doivent respecter les deux autres bras
- Figure 8 : Simulation python pour vérifier nos résultats
- Figure 9 : Première maquette
- Figure 10 : Maquette finale
- Figure 11 : Images 3D des pièces imprimées à l'imprimante 3D
- Figure 12 : Images 2D des pièces découpées à la découpeuse laser
- Figure 13 : Schéma du circuit électrique
- Figure 14 : Schéma cinématique
- Figure 15 : Équations vérifiées par le système
- Figure 16 : Système en boucle fermée avec son correcteur
- Figure 17 : Gabarit et coefficients du PID
- Figure 18 : Equation du PID
- Figure 19 : Influence des termes du PID
- Figure 20 : Pseudo-code de la détection de la bille

Introduction :

Ce rapport de projet détaille le développement d'un système autonome maintenant une bille sur un plateau. L'objectif de ce projet est de concevoir et de réaliser un système esthétique capable de maintenir une bille sur un plateau malgré une faible contrainte imposée à cette bille. Celui-ci devra s'adapter au mouvement d'une bille pour la maintenir sur un plateau qui se déplace grâce à des servomoteurs. Pour atteindre cet objectif, nous avons séparé le projet en 4 axes principaux.

Le développement mécanique a eu pour but de concevoir et de réaliser les différentes pièces permettant de structurer le système et lui permettant de réaliser les différents mouvements indispensables au maintien de la bille sur un plateau. Il doit être assez souple pour réaliser tous les mouvements nécessaires au maintien de la bille sur le plateau mais également assez robuste pour ne pas céder aux contraintes imposées par les mouvements du système. L'enjeu crucial pour cette partie était de réduire au maximum le jeu et les frottements dans la structure pour avoir des mouvements les plus précis possibles. Enfin nous avons fait en sorte de rendre le système le plus esthétique possible pour qu'il soit agréable à l'œil.

La structuration électronique a eu pour but d'installer une alimentation permettant au système d'être suffisamment alimenté en énergie pour qu'il ait la capacité de réaliser les mouvements nécessaires au maintien de la bille sur le plateau. De plus, nous avons fait en sorte de minimiser les branchages nécessaires au fonctionnement du système, ainsi il suffit de brancher deux alimentations pour démarrer notre système. L'enjeu crucial pour cette partie était d'alimenter suffisamment les différents servomoteurs pour permettre aux servomoteurs de déplacer le plateau, de plus, le branchement devait permettre l'équilibre de l'alimentation des différents appareils nécessitant une alimentation et devait être fixé pour nous éviter de faire les branchements à chaque lancement du système. Enfin, nous avons fait en sorte que cette structure soit cachée pour rendre le système plus esthétique.

Le code informatique a eu pour but de réaliser les différentes tâches nécessaires au maintien de la bille sur un plateau. Il permet le suivi de la bille sur le plateau grâce à une caméra, il permet le pilotage des servomoteurs permettant l'inclinaison du plateau pour maintenir la bille sur le plateau, il permet le calcul du mouvement à imposer aux servomoteurs pour suivre un modèle cinématique et il permet de synthétiser le PID nécessaire au maintien de la bille sur le plateau. L'enjeu crucial pour cette partie était d'écrire les différents codes et qu'ils fonctionnent ensemble sans interférer.

L'automatique a eu pour but de calculer le déplacement du plateau nécessaire au maintien de la bille sur celui-ci malgré un mouvement de cette bille. L'enjeu crucial pour cette partie était de s'adapter au mouvement de la bille, aux dimensions de notre plateau et au délai imposé par la caméra.

Ce projet résulte de l'appel d'offre de l'école de l'ESIX qui n'a pas imposé de budget limite. La date du début du projet était le jeudi 26 septembre et la date limite de fin du projet était le jeudi 24 avril. Ce rapport a pour but de montrer les avancées effectuées, les problèmes rencontrés, les décisions prises afin d'atteindre les objectifs fixés.

Partie Mécanique : A-Modélisation du système

Cette partie a été la première à être commencée. En effet un modèle mécanique était nécessaire pour représenter notre système et pour effectuer les tests des autres parties. Nous devons créer une structure capable de maintenir une bille sur un plateau. Nous avons donc commencé par regarder des solutions existantes et une a attiré notre attention.



Figure 1 : Solution existante

Ce système a marqué notre attention car il est plutôt esthétique, il ne nécessite pas l'impression ou la découpe de beaucoup de pièces, nous avons donc décidé d'orienter notre projet vers la réalisation d'un système ressemblant à celui ci-dessus.

Ensuite, nous devons modéliser notre système, nous avons donc commencé par réaliser un schéma cinématique et la loi entrée-sortie de notre système :

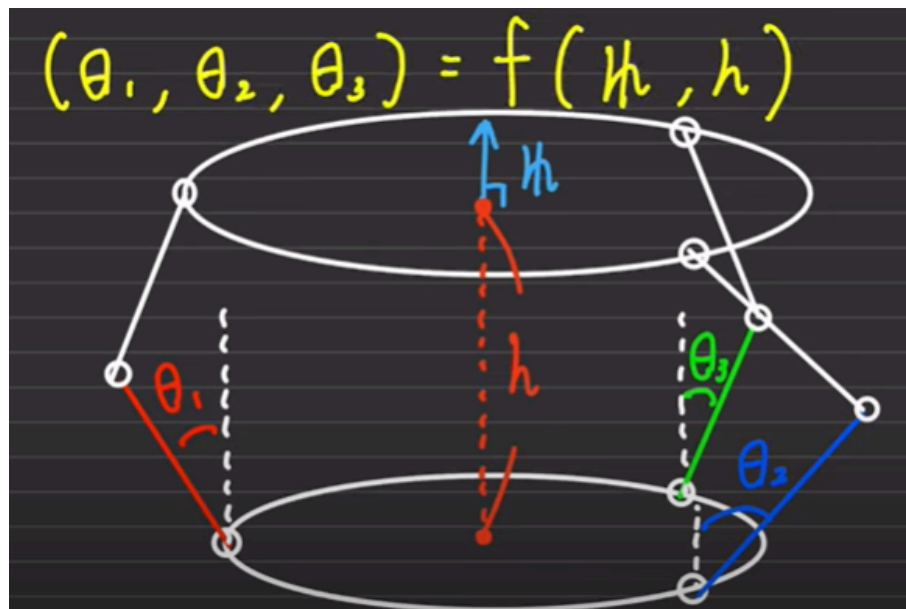


Figure 2 : Schéma cinématique du système

Celle-ci consiste à déterminer l'angle à imposer aux trois servomoteurs en fonction du vecteur normal au plateau et de sa hauteur. Nous devons ensuite déterminer le positionnement des différents servomoteurs pour déplacer le plateau.

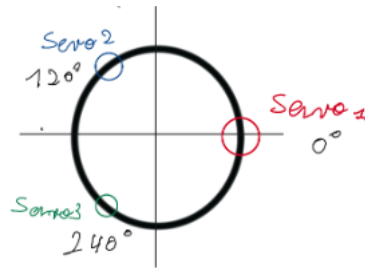


Figure 3 : Schéma du système vu de haut

Nous avons donc décidé de placer nos 3 servomoteurs aux angles 0° , 120° et 240° . Nous avons décidé de faire un plateau de 20 cm de diamètre car ceci nous permet de ne pas faire un plateau et des pièces trop gourmandes en ressources, de plus les appareils disponibles pour ce projet sont en mesure de réaliser des pièces de cette dimension.

Nous devons ensuite déterminer la hauteur à laquelle placer le plateau, pour ce faire, nous devons dans un premier temps déterminer les coordonnées d'un bras. Nous avons donc procédé bras par bras. Pour isoler un bras dans les calculs nous nous plaçons dans un plan par exemple le plan X-Z.

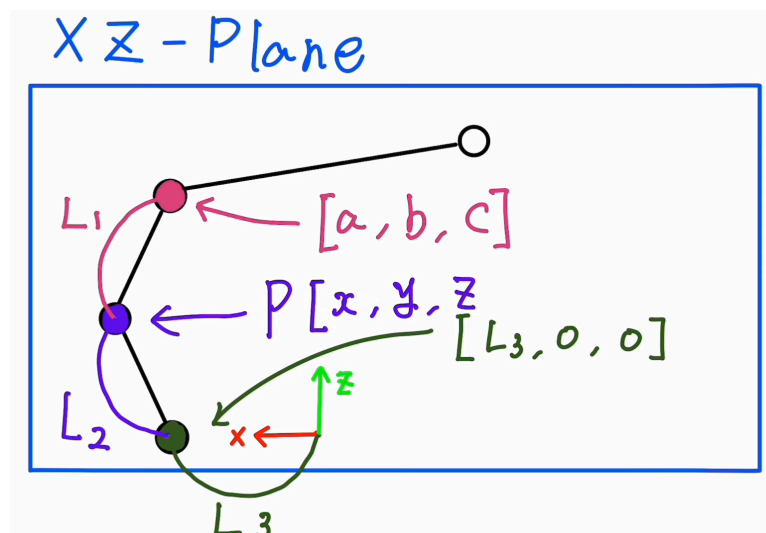


Figure 4 : Analyse cinématique d'un bras du système

Pour déterminer les coordonnées de chaque liaison nous avons utilisé le théorème de pythagore pour déterminer les coordonnées de la rotule $[a, b, c]$.

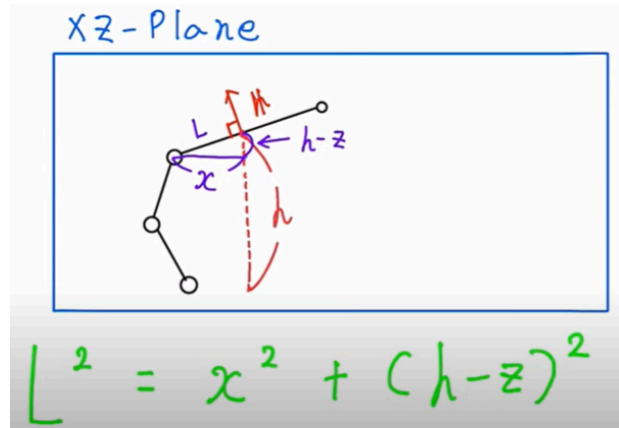


Figure 5 : Détermination des coordonnées des bras du système

Ensuite, nous avons déduit de ce schéma les différentes équations que devait respecter le premier bras.

$$\begin{aligned}
 L^2 &= x^2 + (h-z)^2 \\
 \vec{N} &= [\alpha, \beta, \gamma] \text{ vecteur normal} \\
 \begin{cases} \alpha x + \gamma(z-h) = 0 & \text{équation d'une ligne} \\ L^2 = x^2 + (h-z)^2 & \text{Pythagore} \end{cases} \\
 \left. \begin{aligned} x &= +\sqrt{L^2 - (h-z)^2} \\ z &= h - \frac{\alpha L}{\sqrt{\alpha^2 + \gamma^2}} \end{aligned} \right\} \text{Coordonnées de la rotule plan X;Z}
 \end{aligned}$$

Figure 6 : Détermination des équations que doit respecter le premier bras

Ensuite, nous devons déterminer les équations que devaient respecter les autres bras, nous avons donc effectué les mêmes calculs mais dans un autre plan.

coordonnées des autres rotules :

$$\left. \begin{aligned} x &= \frac{L\gamma}{\sqrt{\alpha^2 + 3\beta^2 + 4\gamma^2 - 2\sqrt{3}\beta\alpha}} \\ y &= \frac{\sqrt{3}L\gamma}{\sqrt{\alpha^2 + 3\beta^2 + 4\gamma^2 - 2\sqrt{3}\beta\alpha}} \\ z &= h + \frac{L3(-\sqrt{3}\alpha + \beta)}{\sqrt{\alpha^2 + 3\beta^2 + 4\gamma^2 - 2\sqrt{3}\beta\alpha}} \end{aligned} \right\} \text{Rotule 2}$$

$$\left. \begin{aligned} x &= -\frac{L\gamma}{\sqrt{\alpha^2 + 3\beta^2 + 4\gamma^2 - 2\sqrt{3}\beta\alpha}} \\ y &= -\frac{\sqrt{3}L\gamma}{\sqrt{\alpha^2 + 3\beta^2 + 4\gamma^2 - 2\sqrt{3}\beta\alpha}} \\ z &= h + \frac{L3(\sqrt{3}\alpha + \beta)}{\sqrt{\alpha^2 + 3\beta^2 + 4\gamma^2 - 2\sqrt{3}\beta\alpha}} \end{aligned} \right\} \text{Rotule 3}$$

Figure 7 : Détermination des équations que doivent respecter les deux autres bras

Nous avons déterminé les coordonnées de la liaison rotule puis nous les avons utilisées pour déterminer les coordonnées de la liaison pivot et nous avons effectué la même chose pour déterminer les coordonnées des servomoteurs. Nous avons donc déterminé la longueur des bras nécessaires au bon fonctionnement du système et nous avons trouvé une longueur de 12 cm pour les bras supérieurs et inférieurs et nous avons trouvé que nous devons placer notre plateau à une hauteur de 14 cm au-dessus des servomoteurs.

Pour vérifier nos résultats, nous avons effectué une simulation python.

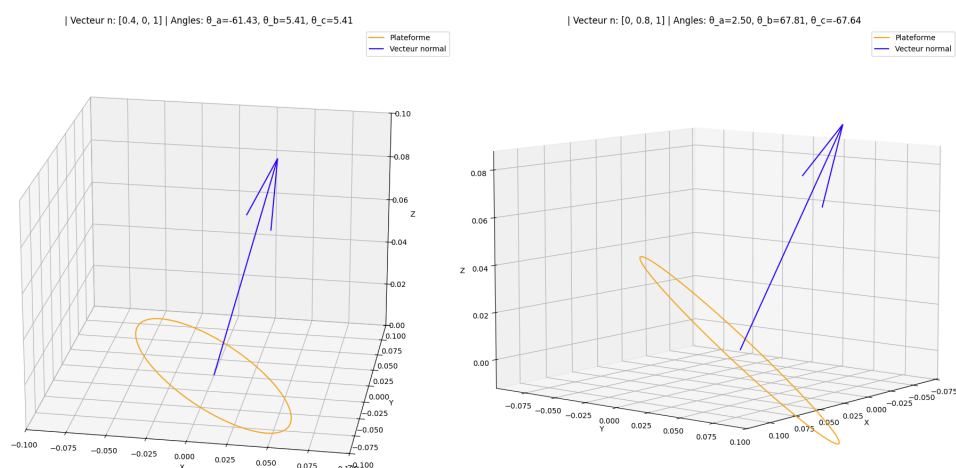


Figure 8 : Simulation python pour vérifier nos résultats

Pour une position de vecteur normal définie au préalable nous observons un comportement similaire entre le système physique(maquette) et la simulation python.

B-Réalisation de la première maquette

Ensuite, nous nous sommes orientés vers une première maquette capable de supporter une bille, permettant le déplacement des servomoteurs et permettant l'inclinaison du plateau.

Nous avons réfléchi aux différentes contraintes mécaniques imposées à notre système et nous en avons conclu que les servomoteurs devaient être fixés, que les bras inférieurs devaient être en liaison encastrement avec les servomoteurs, que les bras inférieurs et les bras supérieurs devaient être en liaison pivot, devaient respecter les calculs effectués précédemment et qu'il devait y avoir une liaison rotule et une liaison encastrement entre le plateau et les bras supérieurs.

Nous avons donc décidé de concevoir une attache fixant le plateau et qui est liée avec les bras supérieurs par l'intermédiaire d'une rotule. Nous devions fixer les servomoteurs donc nous avons décidé de découper un premier socle auquel nous avons fixé les servomoteurs pour limiter le jeu, nous avons également découpé le plateau pour supporter la bille. Nous avons décidé de lier les deux bras supérieur et inférieur avec un roulement à bille pour limiter les frottements et nous avons décidé de fixer le bras inférieur avec les servomoteurs à l'aide de support de servomoteurs.

Pour lier le tout et pour limiter le jeu, nous avons utilisé des vis, des boulons, des entretoises, des rondelles et des tiges filetées et pour permettre le déplacement des servomoteurs, nous avons surélevé le système avec un bloc. En assemblant le tout, nous avons obtenu notre première maquette :

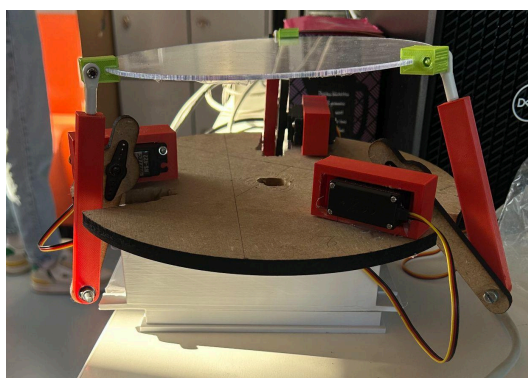


Figure 9 : Première maquette

Grâce à celle-ci, nous avons pu effectuer nos premiers tests sur les servomoteurs, les différentes liaisons et l'inclinaison du plateau. Nous avons ensuite amélioré cette maquette pour permettre l'utilisation de la caméra, nous avons donc imprimé un support de caméra et nous l'avons fixé au socle à l'aide d'un pistolet à colle.

C-Réalisation de la deuxième maquette

Au cours des différents tests, nous nous sommes rendus compte qu'il restait du jeu dans le système et que les déplacements n'étaient pas précis à cause du bloc en dessous, de plus nous devons effectuer les branchements avant chaque manipulation et la maquette n'était pas esthétique.

Nous nous sommes donc orientés vers la création d'une seconde maquette qui sera la maquette finale. Nous avons ajouté un deuxième roulement à bille dans la liaison des bras supérieur et inférieur pour limiter le jeu, nous devons intégrer le système électronique, nous avons donc fait le choix de découper un deuxième socle et des élévateurs permettant de surélever le système et d'implanter le système électronique.

Enfin le système n'était pas très esthétique donc nous avons fait le choix de peindre ou d'imprimer la plupart des pièces en blanc et nous avons peint le logo de l'ESIX. Nous avons repris des pièces de l'ancienne maquette et nous avons assemblé la maquette finale :

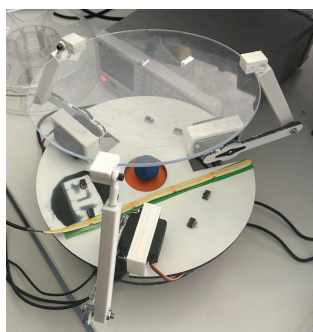


Figure 10 : Maquette finale

Nous avons donc au final une maquette qui limite le jeu et les frottements dans le système, qui est assez souple pour permettre tous les mouvements et assez robuste pour résister aux contraintes imposées par le système.

D-Impression et découpe des pièces de la maquette

Pour concevoir et réaliser les différentes pièces du système, nous avons utilisé deux machines : l'imprimante 3D et la découpeuse laser. Nous avons utilisé l'imprimante 3D lorsque la découpe laser était impossible car la découpeuse laser permettait une réalisation rapide et précise des pièces donc nous pouvions faire des ajustements rapidement et remplacer les pièces en cas de casse. Voici ci-dessous la liste des éléments imprimés avec l'imprimante 3D :

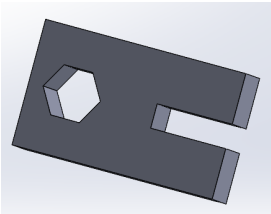
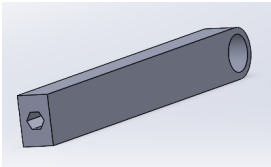
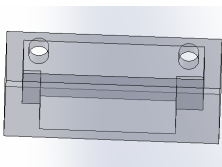
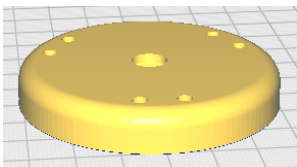
Nom	Attache	Bras supérieur
Image 3D		
Nom	Support servomoteurs	Support caméra
Image 3D		

Figure 11 : Images 3D des pièces imprimées à l'imprimante 3D

Voici ci-dessous la liste des éléments découpés à la découpeuse laser :

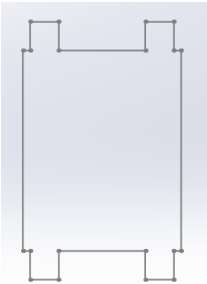
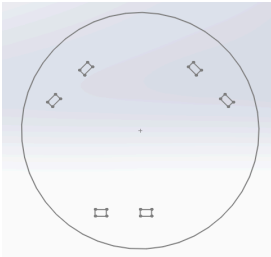
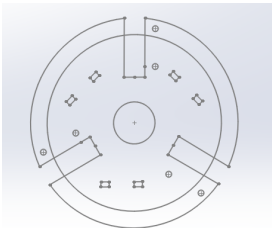
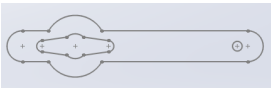
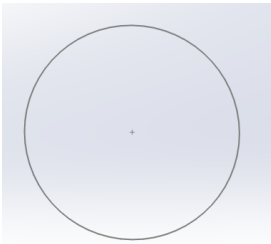
Nom	Elévateur	Socle inférieur
Image 3D		
Socle supérieur	Bras inférieur	Plateau
		

Figure 12 : Images 2D des pièces découpées à la découpeuse laser

Partie Électronique :

A - Evolution du circuit

Au départ nous avons un circuit composé de la pi, du module, du driver et des 3 servos. Au fil du projet, il nous est apparu que le plateau n'était pas bien éclairé, et de ce fait, nous avons ajouté une bande de LED ce qui donne ce schéma ci à la fin :

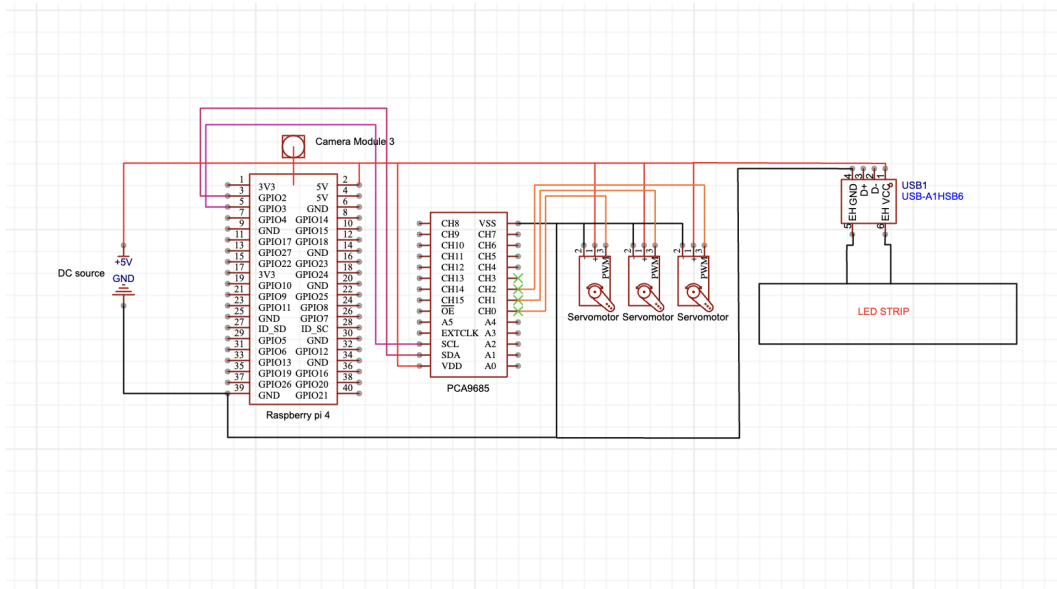


Figure 13 : Schéma du circuit électrique

B - Choix des composants

Matériel utilisé :

Raspberry Pi 4 : Nous devons utiliser une caméra pour l'asservissement. De ce fait, il nous fallait une carte assez puissante pour pouvoir faire tourner openCV mais aussi qui puisse être embarquée.

Raspberry pi camera module 3 : Nous avons choisi ce module caméra car il est parfaitement compatible avec le Raspberry Pi 4 et offre une résolution suffisante pour détecter précisément la position de la bille sur le plateau. Sa compacité et sa facilité d'intégration dans un système embarqué en font un choix idéal pour notre application d'asservissement en temps réel.

Hitec HS-645MG Power servo : Nous avons opté pour ces servomoteurs car ils offrent un couple assez élevé, indispensable pour déplacer le plateau avec précision et rapidité, même sous la contrainte du poids de la bille et des différentes pièces mécaniques.

5 V 4A Power plug : Deux de ces alimentations ont été utilisées pour ce projet. Avec leur intensité maximum à 4A, l'utilisation double nous donne 8A, ce qui est suffisant pour notre projet

Led Strip : Une bande de led non adressable a été préférée. On ne l'alimente qu'en +5 et 0 V pour qu'elle fonctionne. Avec une puissance de 10W/m, on peut même l'alimenter en USB(son mode d'alimentation natif) sans avoir besoin de souder des parties

PCA9685 : Nous avons trois servomoteurs à contrôler. La Pi ne permet que 2 PWM hardware, ce qui n'est pas assez. Donc c'est soit on utilise une PWM software en plus, soit un générateur de pwm externe. Le problème avec les PWM software, c'est qu'ils ne sont ni robustes, ni fiables. De ce fait, l'utilisation d'un PWM hardware était la meilleure. Le PCA, en plus d'avoir sa propre circuiterie de PWM(gain en charge CPU), offre 16 canaux de PWM hardware indépendants. Ce qui est suffisant pour les 3 servomoteurs, et plus encore si besoin.

C - Bilan de puissances

Disponible : 5V 8A

Appareil	Conso estimée (A)	Détail
Servomoteurs (x3)	3	Typique si SG90 (~1A chacun en charge)
PCA9685	0,02	Très faible, négligeable
Raspberry Pi 4	1	Moyenne en fonctionnement normal
LED strip (30 cm)	0,3	Bande LED 5V, 30 cm
Caméra	0,3	Caméra Pi v2 en fonctionnement

Total estimé :

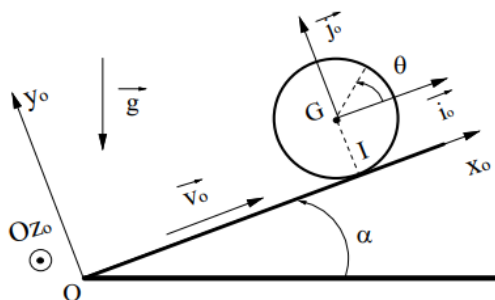
- **Servomoteurs** → 3A
- **PCA9685** → 0.02A
- **Raspberry Pi 4** → 1A (moyenne)
- **LEDstrip** → 0.3A
- **Caméra** → 0.3A

Total $\approx$ 4.6A donc 23W sur 40W disponible

Partie Automatique :

A - Equations différentielles

Nous avons commencé par réfléchir au modèle de notre système et nous avons conclu que le système bille sur plateau peut être assimilé à une bille sur un plan incliné, tel qu'on le voit sur la figure ci-dessous. Nous avons donc modélisé le système et les équations que doit vérifier le système.



Hypothèses :

- 1 La bille roule sans glisser.
- 2 La bille a une masse m et un rayon R
- 3 Le plateau est incliné d'un angle θ par rapport à l'horizontale.
- 4 L'accélération gravitationnelle est g .
- 5 La bille est assimilée à une sphère homogène de moment d'inertie $I = \frac{2}{5}mR^2$.

L'équation du mouvement s'obtient en appliquant :

1. Deuxième loi de Newton (translation) :

$$mg \sin \theta - f_s = ma$$

2. Deuxième loi de Newton (rotation) :

$$f_s R = I \alpha$$

Figure 14 : Schéma cinématique

La force f_s est la force de frottement statique qui empêche le glissement et assure la rotation.

$$f_s R = \frac{2}{5}mR^2 \cdot \frac{a}{R}$$

$$f_s = \frac{2}{5}ma$$

$$mg \sin \theta - \frac{2}{5}ma = ma$$

$$g \sin \theta = a + \frac{2}{5}a$$

$$g \sin \theta = \frac{7}{5}a$$

$$a = \frac{5}{7}g \sin \theta$$

Figure 15 : Équations vérifiées par le système

B - Schéma SIMULINK du procédé

Avec le résultat de l'analyse de la dynamique du système, nous obtenons ainsi l'équation différentielle. Avec celle-ci, nous pouvons établir la fonction de transfert sur Matlab. Nous mettons ensuite la fonction de transfert avec le correcteur en boucle fermée (feedback), ce qui nous permet d'asservir en temps réel notre procédé.

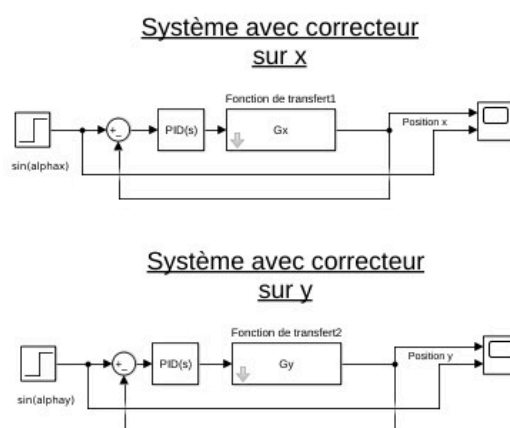


Figure 16 : Système en boucle fermée avec son correcteur

C - Résultats du PID

Nous avons ensuite lancé la simulation de notre système sur Matlab pour obtenir les données de notre PID.

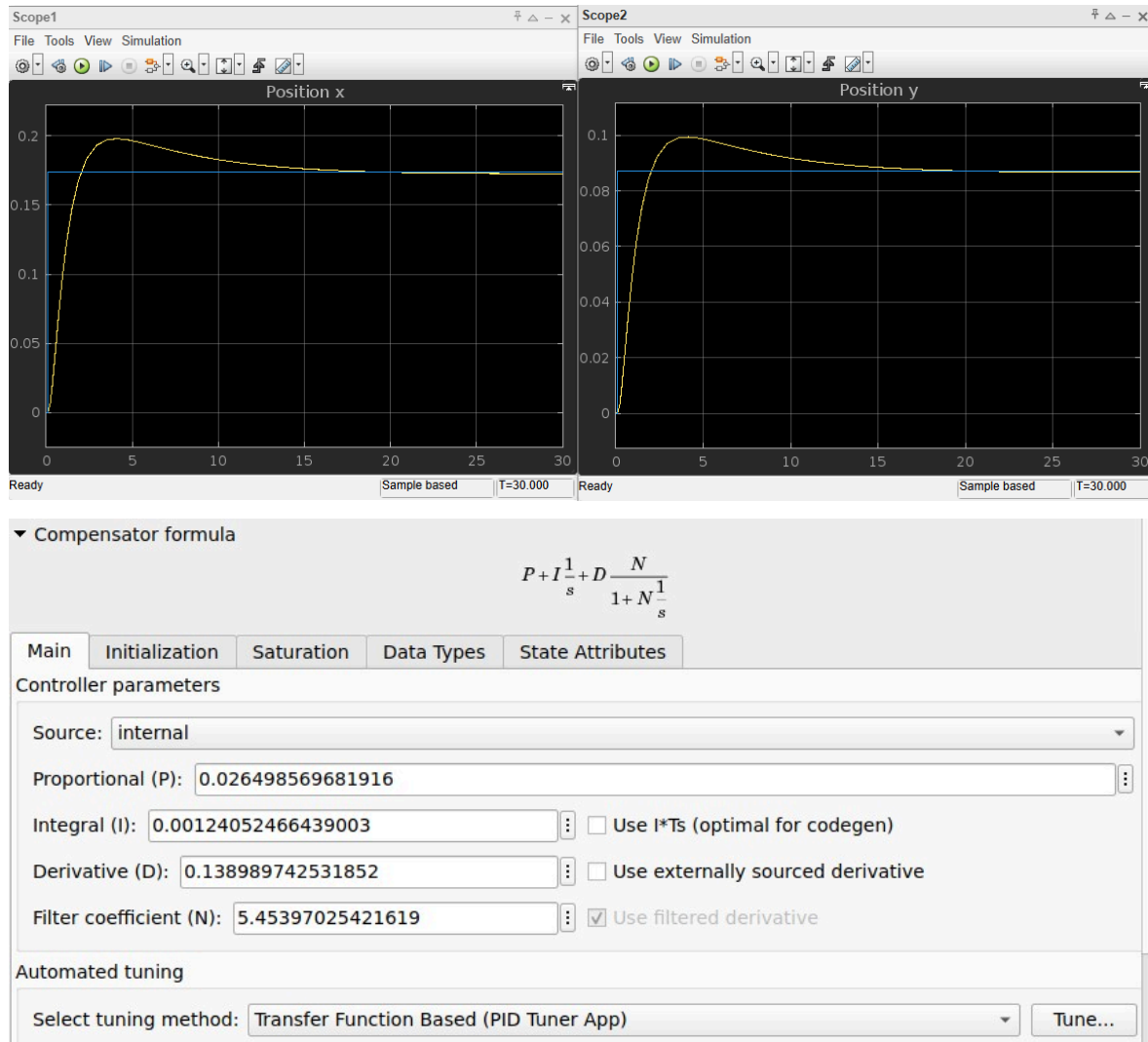


Figure 17 : Gabarit et coefficients du PID

Les résultats qu'on obtient sur Matlab sont ensuite testés sur le système réel. La fonction de transfert est alors remplacée par le système réel, cette implémentation est l'objet de la partie qui va suivre.

D- Implémentation de l'asservissement PID sur Raspberry Pi 4

Dans notre système embarqué basé sur un Raspberry Pi 4, l'asservissement de la position de la bille repose sur **deux boucles de régulation PID** : une pour l'axe X et une autre pour l'axe Y. Chacune de ces boucles corrige la position de la bille en ajustant l'inclinaison de la plateforme via les servomoteurs.

Le correcteur PID (Proportionnel - Intégral - Dérivé) vise à **minimiser l'erreur entre la position cible de la bille et sa position mesurée** en ajustant dynamiquement l'orientation du plateau :

$$u(t) = K_p \cdot e(t) + K_i \cdot \int e(t)dt + K_d \cdot \frac{de(t)}{dt}$$

où :

- $e(t) = \text{position_cible} - \text{position_mesurée}$
- K_p, K_i, K_d sont les gains du PID
- $u(t)$ est la commande envoyée (par exemple un angle d'inclinaison ou une position de servo).

Figure 18 : Equation du PID

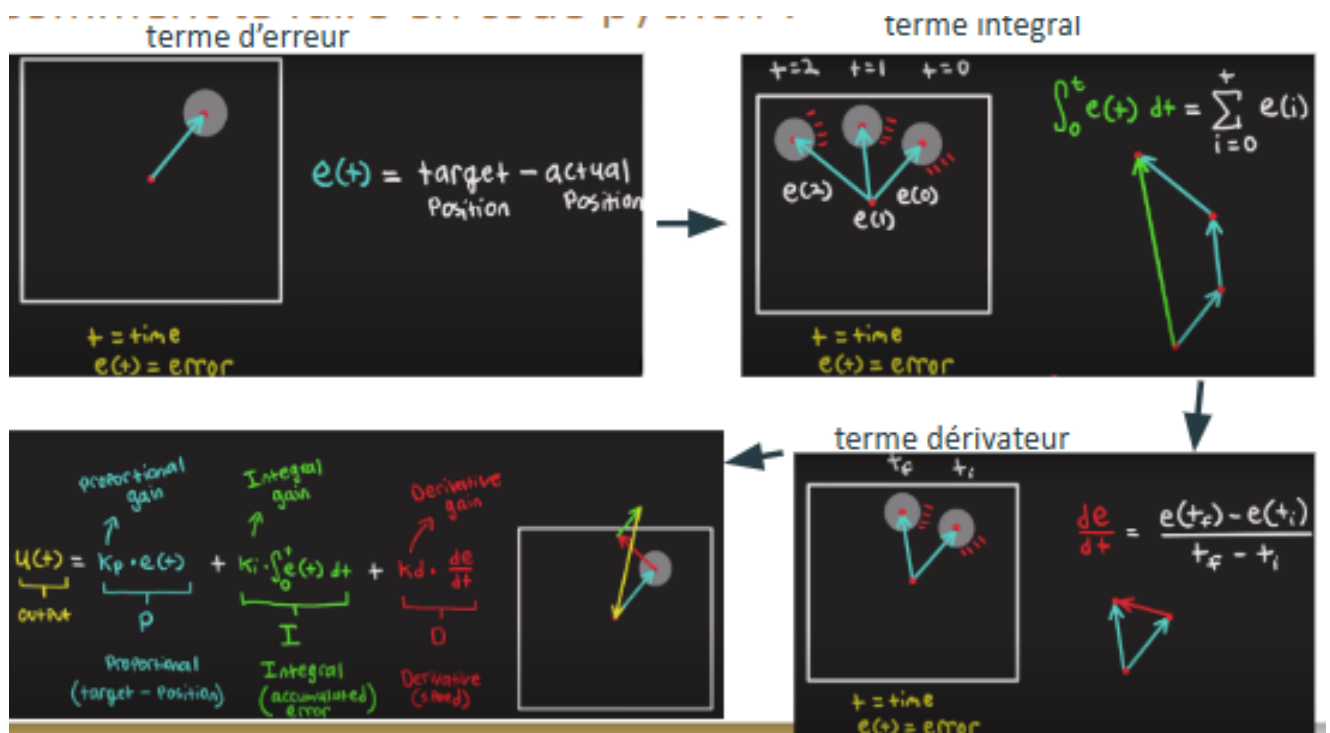


Figure 19 : Influence des termes du PID

Terme d'erreur - $e(t)$

Ce terme correspond à la différence entre la position cible et la position actuelle:

$$e(t) = \text{target} - \text{actual}$$

Il indique dans quelle direction et avec quelle intensité il faut corriger. La flèche représente l'erreur.

Terme intégral - $\int e(t)dt$

Ce terme additionne les erreurs au cours du temps pour corriger les écarts persistants. L'intégrale est souvent calculée de façon discrète :

$$\int_0^t e(\tau)d\tau \approx \sum_{i=0}^t e(i)$$

Cela permet de supprimer une erreur résiduelle constante (offset).

Terme dérivatif - $\frac{de(t)}{dt}$

Ce terme évalue la vitesse de variation de l'erreur :

$$\frac{de(t)}{dt} = \frac{e(t) - e(t-1)}{dt}$$

Il permet d'anticiper les changements rapides et d'amortir les oscillations

Synthèse PID : Les trois termes sont combinés pour calculer la commande :

$$u(t) = K_p \cdot e(t) + K_i \cdot \int e(t)dt + K_d \cdot \frac{de(t)}{dt}$$

Partie Informatique :

Pour la partie informatique, nous avons utilisé Python comme langage de programmation principal, car il est particulièrement adapté à la Raspberry Pi 4 grâce à sa richesse en bibliothèques et à sa simplicité d'usage.

Nous avons eu recours aux bibliothèques suivantes :

- **Picamera 2** : acquisition d'images via le module caméra.
- **OpenCV** : traitement d'images en temps réel.
- **Adafruit Servo Kit** : commande des servomoteurs.
- **numpy** : calculs mathématiques et manipulation de tableaux.
- **matplotlib** : visualisation de données pour le débogage et la simulation.

A-Détection de la bille (Vision par ordinateur)

Le principal défi informatique concernait la détection précise et en temps réel de la position de la bille sur le plateau.

La méthode utilisée s'est déroulée comme suit :

Acquisition d'images en temps réel via Picamera 2.

Traitement d'image avec OpenCV :

- Conversion en niveaux de gris.
- Application d'un seuillage.
- Détection des contours circulaires.
- Extraction des coordonnées (x, y) de la bille.

Pour faciliter la détection, nous avons choisi une bille bleue, plus facilement identifiable dans l'encodage BGR utilisé par la caméra. Une mauvaise interprétation initiale (RGB au lieu de BGR) nous a fait perdre du temps au début du projet.

Le raisonnement a ensuite évolué vers une détection en espace HSV, en définissant deux plages de teinte rouge pour créer un masque efficace, comme illustré dans le pseudo-code ci-dessous :

INITIALISATION

Configurer la caméra Picamera2 (résolution 480x480, 50 FPS)

Définir les plages HSV pour la couleur rouge :

- Plage 1 : (H1_min, S1_min, V1_min) à (H1_max, S1_max, V1_max)
- Plage 2 : (H2_min, S2_min, V2_min) à (H2_max, S2_max, V2_max)

BOUCLE TANT QUE

```

image ← capture caméra
image_HSV ← conversion en HSV
masque ← application des deux plages HSV pour le rouge (une seule pour le bleu)
contours ← détection contours

si contours non vides et rayon > seuil :
    (x, y), r ← coordonnées du cercle englobant le plus grand contour
    x_norm ← (x - largeur/2) / (largeur/2)
    y_norm ← (y - hauteur/2) / (hauteur/2)
    envoyer_position(x_norm, y_norm)
sinon :
    afficher "Aucune bille détectée"
    
```

B-Structure logicielle et architecture modulaire

Le projet a été structuré en plusieurs modules Python :

- `cinematique_VF2.py` : Ce module calcule les angles des servomoteurs à partir de l'inclinaison souhaitée (cinématique inverse)
- `class_PID.py` : Ce module PID est utilisé pour le calcul du vecteur d'inclinaison (θ , ϕ) à partir de la position d'erreur.
- `main.py` (ou fichier principal) : Ce module orchestre l'ensemble du processus, en incluant la détection de la bille, la commande PID, la conversion en angles, et l'envoi des commandes aux servomoteurs via Adafruit

Pipeline fonctionnel :

Détection de la position de la bille (`find_ball`) :

- Ce thread utilise Picamera 2 pour capturer une image en continu.
- L'image est convertie en espace HSV, puis deux masques de teinte rouge sont appliqués.
- Les contours sont extraits, et si un objet suffisamment grand ($\text{rayon} > 20\text{px}$) est détecté, sa position (x , y) est enregistrée sous forme relative (centrée autour de (0, 0)).
- Cette position est stockée dans une variable partagée, protégée par un verrou (`position_lock`).

Asservissement et pilotage (control_motors) :

- Ce thread lit la position de la bille.
- S'il n'y a aucune bille détectée, le robot reste en position initiale.
- Sinon, la position actuelle est comparée à la consigne (goal), et un PID 2D calcule le vecteur d'inclinaison désiré :

theta : direction d'inclinaison (angle en degrés).

phi : amplitude d'inclinaison (proportionnelle à l'erreur).

Commande des moteurs (control_t_posture) :

- Reçoit le vecteur (theta, phi) et la hauteur Pz.
- Calcule le vecteur normal du plateau à partir de theta, phi (conversion sphérique → cartésien).
- Passe ce vecteur à la fonction de cinématique inverse kinema_inv() du robot.
- Envoie les angles retournés aux 3 servos via Adafruit ServoKit.

C-Configuration de l'environnement Linux

Un travail important a été réalisé sur la mise en place de l'environnement de développement sur la Raspberry Pi :

- Système utilisé : Raspberry Pi OS Bookworm (Debian 12).
- Installation des dépendances :

```
sudo apt update
sudo apt install python3-pip python3-opencv libatlas-base-dev
pip install numpy opencv-python picamera2 adafruit-circuitpython-servokit matplotlib
```
- Connexion SSH activée pour le développement à distance.
- Utilisation de TightVNC pour l'accès graphique en cas de besoin.
- Structure des fichiers :

```
/home/pi/bille_plateau_project/
├── main.py
├── class_PID.py
└── cinématique_VF2.py
```

Difficultés rencontrés :

Mécanique :

- La prise en main de Solidworks a pris plus de temps que prévu ce qui a retardé le lancement du projet.
- Le jeu et les frottements dans le système étaient des paramètres dont il fallait tenir compte dans la réalisation des pièces du système, ce qui nous contraignait dans la réalisation de celles-ci.
- La commande de pièces nécessaires à la structuration du système et la livraison de celles-ci nous ont retardé dans la réalisation de la maquette finale du système.
- Nous n'avons pas effectué la formation de la découpeuse laser, nous étions donc dépendant des autres membres de la classe pour réaliser certaines pièces de notre système, ce qui nous a également retardé.

Electronique :

- Nous avons dû rajouter une alimentation de plus pour pouvoir avoir assez de puissance pour faire fonctionner tout le circuit.
- Les servomoteurs défaillants au début du projet ont rendu difficile la possibilité d'avancer sur le fonctionnement du circuit dans sa globalité
- Nous avons dû choisir une bande LED qui ne prend pas beaucoup de puissance au générateur avec une connectique compatible avec la raspberry.
- Une modification de la structure mécanique du système était nécessaire pour pouvoir intégrer toute l'électronique de manière esthétique et fonctionnelle.

Automatique :

- Nous avons eu des difficultés à mettre en oeuvre un correcteur PID sur Python
- Le calcul des gains proportionnels, intégraux, dérivés, du correcteur PID sur le système réel a posé problème, car la fonction de transfert simulé sur Matlab ne représente pas la fonction de transfert exacte du système réel
- Le problème majeur de cette partie était la mise en oeuvre d'un avance de phase, pour réduire le retard dû à la caméra.

Informatique :

- Nous avons eu des difficultés à prendre en main la connexion SSH avec la raspberry pi, ce qui a amené des soucis de connexions à la raspberry lorsque l'on souhaitait effectuer des tests.
- Nous étions limités par les librairies disponibles pour la détection d'objet sur python.
- Nous avons eu des difficultés pour déboguer nos codes avec python.

Conclusion :

Nous avons vu que notre projet est perfectible, une meilleure anticipation de certaines tâches, de certaines commandes auraient pu nous permettre de finir à temps le projet.

Néanmoins, ceci n'est pas une fatalité, ce projet nous aura appris à être plus précis dans la planification et la réalisation des tâches, concernant un point de vue gestion de projet.

D'un point de vue technique, ce projet nous aura appris à concevoir et réaliser des pièces 3D et 2D et à les imprimer avec une imprimante 3D ou à les découper avec une découpeuse laser, à réfléchir à un assemblage respectant différentes contraintes imposées à un système et lui permettant de réaliser certains mouvements sans se briser.

Nous avons également appris à alimenter un système autonome, pour qu'il ait assez d'énergie pour réaliser les tâches qu'on lui demande. Nous avons réfléchi à comment alimenter les différents composants du système et que l'alimentation soit intégrée au système.

Nous avons appris à écrire du code nous permettant de contrôler simultanément des servomoteurs, de suivre une bille grâce à un traitement numérique des données transmises par une caméra, d'écrire un PID capable de réagir aux données reçus en donnant les consignes appropriées et à calculer les commandes nécessaires au maintien d'une bille sur un plateau en suivant un schéma cinématique établi.

Enfin, nous avons appris à réaliser un PID capable de réagir aux données qu'il reçoit et à envoyer la commande nécessaire à des servomoteurs pour permettre le maintien d'une bille sur le plateau.

D'un point de vue relationnel, ce projet nous a appris à nous coordonner pour mener à bien un projet, il nous a appris à nous répartir les tâches pour être plus efficace lors de la réalisation d'un projet.

Annexes

Datasheets des composants :

<https://www.raspberrypi.com/products/raspberry-pi-4-model-b/specifications/>

<https://datasheets.raspberrypi.com/camera/camera-module-3-product-brief.pdf>

<https://media.digikey.com/pdf/Data%20Sheets/Hi-Tech/HS-645MG.pdf>

<https://m.media-amazon.com/images/I/71wZbgCB0WL.jpg>